

CMPE 492

Performance Analysis of Secure Container Runtimes
on Kubernetes

Mustafa Ocak

Halil İbrahim Kasapoğlu

Advisor:

Bahri Atay Özgövde

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. Broad Impact	1
1.2. Ethical Considerations	1
2. PROJECT DEFINITION AND PLANNING	3
2.1. Project Definition	3
2.2. Project Planning	3
2.2.1. Project Time and Resource Estimation	3
2.2.1.1. Research Phase	3
2.2.1.2. Implementation Phase (8 weeks)	4
2.2.1.3. Success Criteria	5
2.2.2. Risk Analysis	6
2.2.3. Team Work	7
3. RELATED WORK	8
3.1. Related Articles	8
3.1.1. Performance and Isolation Analysis of runC , gVisor , and Kata Containers	8
3.1.2. Comparison of Container Runtimes in Kubernetes	8
3.1.3. Architecture and Performance of Contemporary Isolation Platforms	9
3.1.4. Security-Performance Trade-offs of Kubernetes Container Runtimes	9
3.1.5. Performance Guidelines by gVisor	9
3.1.6. Additional Works Considered During the Project	9
3.1.6.1. gVisor : A Lightweight Security Solution	10
3.1.6.2. Cilium : eBPF-Powered Kubernetes Networking and Security	10
3.1.6.3. Falco : Runtime Security for Kubernetes	10
3.1.6.4. Aqua Security : End-to-End Container Security	10
4. Methodology	11
4.1. Kubernetes Cluster Setup	11

4.2.	Container Runtime Environment Setup	11
4.3.	Load Testing with Locust	12
4.4.	Performance Metrics Collection	12
4.5.	Benchmarking and Data Analysis	13
5.	REQUIREMENTS SPECIFICATION	14
5.1.	Scope and Goals	14
5.2.	Functional Requirements	14
5.2.1.	Runtime Configuration	14
5.2.2.	Benchmarking Configuration	14
5.2.3.	Testing Setup and Execution	15
5.2.4.	Documentation and Reporting	15
5.3.	Nonfunctional Requirements	15
5.3.1.	System Requirements	15
6.	DESIGN	17
6.1.	Information Structure	17
6.2.	Information Flow	18
6.2.1.	Activity Diagram	18
6.2.2.	Sequence Diagram	19
6.3.	System Design	19
6.3.1.	Container Runtime Architectures	19
6.3.2.	Overall System Design	21
7.	IMPLEMENTATION AND TESTING	22
7.1.	Implementation	22
7.1.1.	Microservice Architecture and Stack	22
7.1.2.	Container Runtime Setup	23
7.2.	Testing	23
7.2.1.	Test Scenarios	23
7.2.2.	Test Execution	24
7.3.	Deployment	25
7.3.1.	Kubernetes Configuration	25
7.3.1.1.	Container Runtime Configuration	25

7.3.1.2. Microservice Deployment	26
8. RESULTS	27
8.1. CPU Consumption	27
8.2. Memory Usage	28
8.3. Response Time Comparison	28
8.4. Container Startup Time	29
8.5. Throughput at Low Load	29
8.6. Throughput at High Load	30
8.7. Summary of Results	30
9. CONCLUSION	32
REFERENCES	34
APPENDIX A: SAMPLE APPENDIX	36
A.0.1. *	36

1. INTRODUCTION

1.1. Broad Impact

The rapid adoption of container orchestration tools like Kubernetes has transformed the way organizations deploy and manage applications. However, Kubernetes faces critical security and operational challenges, particularly when handling sensitive workloads in sectors such as finance, healthcare, and cloud services. This project, **“Performance Analysis of Secure Container Runtimes on Kubernetes”**, aims to address these challenges by enhancing Kubernetes’ orchestration mechanisms to improve security and reliability.

Containerized environments are the backbone of modern cloud-native applications. Traditional container runtimes like `runc` prioritize performance over security, leaving room for improvement in isolation and resilience against exploits. This project evaluates and benchmarks secure container runtimes `runsc` (gVisor) and `Kata Containers` against the widely used `runc` in a Kubernetes setting.

By analyzing runtime performance under realistic microservice workloads and providing a reusable testing suite, the project will help improve security and performance in Kubernetes deployments. It will also contribute to the broader ecosystem by offering insights into the trade-offs between performance and security in container runtimes for sensitive use cases.

1.2. Ethical Considerations

Given that this project focuses on performance benchmarking, the primary ethical considerations are:

- **Fairness in Benchmarking:** We ensure that all container runtimes and config-

urations are tested under the same conditions to provide fair, unbiased results, enabling meaningful comparisons.

- **Transparency:** Test methodologies, datasets, and configurations will be clearly documented and shared publicly to ensure the reproducibility of results and foster trust in the outcomes.
- **Resource Efficiency:** The performance tests will consider resource usage, ensuring that the benchmarks do not unnecessarily waste computational resources, minimizing environmental impact.
- **Data Integrity:** Any data collected during benchmarking will be handled responsibly, ensuring its accuracy and safeguarding against misinterpretation of results.

The project aims to provide clear, reliable performance insights while maintaining fairness, transparency, and minimal resource consumption.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

The objective of this project is to analyze the performance of different container runtimes (**runc**, **runsc**, and **Kata Containers**) in a Kubernetes environment, focusing on the trade-offs between performance and security. The primary focus is runtime performance evaluation through key metrics such as CPU usage, memory consumption, response time, and throughput, while acknowledging the impact of enhanced isolation on performance.

2.2. Project Planning

2.2.1. Project Time and Resource Estimation

The project will span one semester, with key dates being the midterm report (11th November) and the final presentation (13th January). The project can be divided into two main phases: Research and Implementation.

2.2.1.1. Research Phase. The research phase will focus on understanding the security concerns of Kubernetes and its core features. All findings will be supported by practical work and documented in a GitHub repository.

(i) **VM and Container Environment Setup (1 week)**

- Initial research on VM and container setup.

(ii) **Kubernetes Cluster Setup (2 weeks)**

- Investigate the main components of Kubernetes.
- Set up and configure a Kubernetes cluster.
- Run load tests using the Locust tool to evaluate performance.

(iii) **Investigating Storage Options on Kubernetes (1 week)**

- Explore and document available storage options in Kubernetes.

(iv) **Investigating Container Runtimes (2 weeks)**

- Study and analyze the most used container runtimes, mainly `runc`, `runsc` and `katacontainers`.

2.2.1.2. Implementation Phase (8 weeks).

(i) **Apache Load Testing with Kubernetes and VM Setup (Week 1-2)**

- Week 1:
 - Set up a virtual machine and deploy an Apache web server.
 - Install Apache Benchmark (`ab`) tool and configure it to generate load on the VM.
 - Conduct initial load tests on the VM and collect performance metrics (response time, throughput, CPU, and memory usage).
- Week 2:
 - Set up a Kubernetes cluster and deploy an Apache web server as a pod.
 - Perform load tests using the Apache Benchmark tool on the Kubernetes-deployed Apache web server.
 - Compare the performance results of Apache running on the VM and Kubernetes, focusing on differences in resource usage and response times.

(ii) **Cluster Setup for Different Runtime Environments (Week 3-5)**

- Week 3:
 - Install and configure the default container runtime, `runc`, on the Kubernetes cluster.
 - Deploy a microservice-based application on the cluster using `runc` as the runtime.
 - Verify the deployment and ensure all services are functioning correctly.
- Week 4:
 - Install and configure `gVisor` as the container runtime on a separate Kubernetes cluster.
 - Deploy the same microservice-based application on the cluster using

gVisor.

- Verify deployment and perform initial testing to ensure stability.

- Week 5:

- Install and configure **Kata Containers** as the container runtime on another Kubernetes cluster.
- Deploy the same microservice-based application using **Kata Containers**.
- Verify deployment and check for compatibility or specific issues related to Kata Containers.

(iii) **Locust Load Testing and Performance Analysis (Week 6-8)**

- Week 6:

- Set up the Locust load testing framework.
- Design and configure test scenarios to simulate concurrent user requests on the microservice-based application.
- Run Locust tests on the Kubernetes cluster using **runc** and collect performance metrics.

- Week 7:

- Run Locust tests on the Kubernetes cluster using **gVisor**.
- Collect and document metrics such as response time, throughput, CPU, and memory usage during the load tests.
- Compare the results with those obtained from the **runc** tests.

- Week 8:

- Run Locust tests on the Kubernetes cluster using **Kata Containers**.
- Collect and document performance metrics.
- Analyze and compare the results across all three runtime environments.
- Prepare detailed performance analysis and document the findings.

2.2.1.3. Success Criteria. This project will be considered successful if it meets the following conditions:

- (i) Team members demonstrate the ability to set up and use Kubernetes as a con-

tainer orchestration platform, showing proficiency in creating and managing Kubernetes environments, deploying applications, and effectively utilizing its core features.

- (ii) A comprehensive performance analysis of container runtimes (**runc**, **runsc**, and **Kata Containers**) is conducted, focusing on key metrics such as CPU usage, memory consumption, startup time, and response time under different workloads.
- (iii) Trade-offs between performance and security are identified, with detailed insights provided on how enhanced isolation in runtimes like **runsc** and **Kata Containers** impacts system overhead and resource utilization.

2.2.2. Risk Analysis

- (i) **Technical Complexity of Kubernetes and Security:**

Risk: Kubernetes and container security mechanisms are complex and evolving.

Mitigation: Leverage community resources, documentation, and expert consultations.

- (ii) **Resource Constraints:**

Risk: Limited access to cloud environments for large-scale testing.

Mitigation: Utilize free-tier cloud services and local virtual machines for initial testing.

- (iii) **Evolving Security Threats:**

Risk: Emerging security vulnerabilities during the project.

Mitigation: Stay updated on security trends and adjust accordingly.

- (iv) **Performance Variability:**

Risk: The performance of different container runtimes may vary significantly depending on workload types and cluster configurations, making consistent benchmarking difficult.

Mitigation: Use standardized test scenarios and repeat tests multiple times to reduce variability and ensure reliable results.

- (v) **Time Management:**

Risk: The project involves multiple phases (performance analysis, runtime com-

parison, and preliminary security research), which may lead to time overruns.

Mitigation: Maintain a clear timeline with defined milestones and regularly review progress to stay on track.

2.2.3. Team Work

Both members of our team are fully engaged in the project, actively conducting research and exploring new repositories, articles in secure container orchestration, and Kubernetes. We approach all phases collaboratively, from initial research to repository management and presentation development.

3. RELATED WORK

In recent years, numerous studies have explored the performance and security trade-offs of container runtimes, particularly in Kubernetes environments. Below, we summarize relevant works that provide valuable insights into the evaluation of container runtimes such as `runC`, `gVisor`, and `Kata Containers`.

3.1. Related Articles

3.1.1. Performance and Isolation Analysis of `runC`, `gVisor`, and `Kata Containers`

In [1], the authors investigate the performance characteristics and isolation capabilities of `runC`, `gVisor`, and `Kata Containers`. They evaluate critical dimensions such as startup time, system call interception, resource density, and isolation. The results highlight that while `runC` and `Kata Containers` achieve minimal performance overhead, `gVisor` experiences significant performance degradation in I/O-heavy scenarios due to its system call interception mechanism. Nevertheless, `gVisor` provides superior isolation, making it a viable option where security is paramount.

3.1.2. Comparison of Container Runtimes in Kubernetes

The work in [2] focuses on comparing traditional container runtimes like `containerd` with newer, more secure alternatives such as `Kata Containers`, `Firecracker`, and `gVisor`. The authors deploy these runtimes in a Kubernetes testbed and measure key performance metrics, including container startup time. Their findings indicate that `gVisor` offers the fastest startup time and robust security, while `Firecracker` demonstrates higher startup latency due to its reliance on lightweight virtual machines.

3.1.3. Architecture and Performance of Contemporary Isolation Platforms

The study in [3] examines various isolation platforms, including containers, secure containers, hypervisors, and unikernels. The authors conduct a detailed evaluation of performance overheads and isolation capabilities, revealing that containers offer near-native performance but lack robust isolation. Secure containers like **Kata Containers** and **gVisor** provide stronger isolation at the cost of additional overheads. Unikernels, however, achieve the highest degree of isolation, albeit with limited practicality in some scenarios.

3.1.4. Security-Performance Trade-offs of Kubernetes Container Runtimes

In [4], the authors conduct a systematic comparison of **runC**, **gVisor**, and **Kata Containers** within a Kubernetes environment. By deploying real-world applications, the study demonstrates that **runC** outperforms the secure alternatives by up to 5x in performance. **gVisor** exhibits faster container deployment times compared to **Kata Containers**, while the latter excels in execution speed. The authors emphasize the plug-and-play nature of secure runtimes in Kubernetes, albeit with significant performance penalties.

3.1.5. Performance Guidelines by gVisor

The official performance guide by **gVisor** [5] provides an in-depth look at its architecture and practical advice for optimizing workloads. It discusses the trade-offs of syscall interception and strategies to mitigate performance overhead while maintaining high isolation standards.

3.1.6. Additional Works Considered During the Project

Below are additional works that inspired and guided our understanding of Kubernetes security during the mid-course of the project:

3.1.6.1. gVisor: A Lightweight Security Solution. gVisor [6] is a user-space kernel designed to provide isolation for containerized workloads by intercepting system calls and implementing an alternative kernel interface. Its lightweight design makes it ideal for reducing the attack surface while maintaining performance for microservices.

3.1.6.2. Cilium: eBPF-Powered Kubernetes Networking and Security. Cilium [7] leverages eBPF (Extended Berkeley Packet Filter) technology to enhance networking and security within Kubernetes. By enabling dynamic enforcement of network policies at the kernel level, Cilium ensures secure communication between pods while offering observability and performance optimizations.

3.1.6.3. Falco: Runtime Security for Kubernetes. Falco [8], an open-source runtime security tool, focuses on detecting unexpected behavior in Kubernetes workloads. It uses kernel-level instrumentation to monitor system calls and applies rule-based logic to identify potential security breaches in real time.

3.1.6.4. Aqua Security: End-to-End Container Security. Aqua Security [9] provides a comprehensive solution for securing containerized applications, including vulnerability scanning, runtime protection, and compliance enforcement. Aqua's integration with Kubernetes emphasizes the need for end-to-end security in cloud-native environments.

4. Methodology

This chapter outlines the methodology followed in setting up the Kubernetes cluster, configuring different container runtimes, performing load tests, and collecting performance metrics. The primary objective was to assess the trade-offs between performance and security across different runtime environments.

4.1. Kubernetes Cluster Setup

A Kubernetes cluster was set up using `kubeadm`, a widely used tool for bootstrapping Kubernetes. The setup involved initializing a master node and joining multiple worker nodes to form a functioning cluster. A container network interface (CNI) plugin was deployed (Calico) to enable communication between pods. The cluster was verified to ensure all nodes and core components were functioning correctly.

4.2. Container Runtime Environment Setup

To conduct a comparative analysis of container runtimes, the following environments were configured:

- (i) **runc**: The default runtime for most container platforms, known for its minimal overhead and high performance.
- (ii) **runsc (gVisor)**: A runtime focused on enhanced security, which provides isolation by intercepting and emulating system calls in user space.
- (iii) **Kata Containers**: A runtime that offers strong isolation by running containers inside lightweight virtual machines, combining the benefits of VMs and containers.

Each runtime environment was set up separately on dedicated namespaces to prevent interference and ensure accurate performance measurements. Configuration changes were made to runtime classes and Kubernetes to switch between runtimes

as needed. After each runtime was configured, deployments were verified by running sample applications to confirm compatibility.

4.3. Load Testing with Locust

To evaluate runtime performance, load tests were conducted using **Locust**, a scalable load testing tool. A microservice-based application was deployed on the Kubernetes cluster, and the same workload was tested across all runtime environments.

Locust was used to simulate concurrent user requests over a specified period, generating key metrics such as:

- Average and median response time
- Request throughput
- Error rate
- Latency

The load tests were repeated multiple times to ensure consistency and minimize variability in the results. Each run was at least 5 minutes long.

4.4. Performance Metrics Collection

Performance metrics were collected using **Prometheus** and **Kube State Metrics**, which provided insights into the resource consumption of pods and nodes during the tests. Key metrics included:

- CPU usage by pods and nodes
- Memory consumption by pods and nodes
- Disk I/O and network usage

Custom bash scripts were written to automate the deployment of workloads,

execution of Locust tests, and retrieval of metrics from Prometheus. These scripts helped streamline the testing process and ensure consistency across different runtime environments.

4.5. Benchmarking and Data Analysis

After collecting the performance metrics, data analysis was performed to compare the runtime environments. Python scripts were used to process the data and generate visualizations, including box plots and bar charts, to highlight key differences in:

- Response time and throughput
- Resource consumption (CPU and memory)
- Performance variability under different load conditions

The analysis focused on identifying the trade-offs between performance and security. While `runc` demonstrated superior performance with minimal overhead, `runsc` and `Kata Containers` provided enhanced isolation at the expense of increased resource consumption and response time.

5. REQUIREMENTS SPECIFICATION

5.1. Scope and Goals

- Evaluate the performance of container runtimes: *runc*, *runsc* (gVisor), and *Kata Containers*, in Kubernetes environments.
- Publish setup configurations to enable reproducibility of experiments.
- Focus on performance evaluation while addressing security trade-offs.

5.2. Functional Requirements

5.2.1. Runtime Configuration

- **Configure runc:** Install and configure *runc* as the default container runtime in the Kubernetes cluster.
- **Configure runsc (gVisor):** Install and configure *runsc* as an alternative runtime in the cluster.
- **Configure Kata Containers:** Install and configure *Kata Containers* as an alternative runtime in the cluster.

5.2.2. Benchmarking Configuration

- **Microservice Benchmarking Setup:** Deploy a standardized microservice stack on Kubernetes. Use *Locust* for simulating user traffic and generate load across runtimes.
- **Performance Metrics Collection:** Collect throughput, latency, resource usage (CPU, memory), and startup times via *Prometheus* and *Kubernetes Metric Server*.

5.2.3. Testing Setup and Execution

- **Kubernetes Cluster Setup:** Configure a Kubernetes cluster with multiple runtimes and namespaces to facilitate load testing and runtime switching.
- **Load Testing Execution:** Execute load tests using *Locust* and measure runtime performance under varying loads.
- **Data Logging and Analysis:** Record all performance data, including CPU, memory consumption, throughput, and latency, and analyze results across different workloads using Prometheus.

5.2.4. Documentation and Reporting

- **Reproducibility Documentation:** Provide clear configuration documentation (including YAML files) for Kubernetes deployment, benchmarking setup, and runtime configurations.
- **Performance Reporting:** Present performance results in visual formats (graphs/charts) for transparency and share them for public access.

5.3. Nonfunctional Requirements

5.3.1. System Requirements

- Kubernetes version 1.31 or higher.
- Containerd (latest version) as the Container Runtime Interface (CRI).
- Prometheus and Kubernetes Metric Server for monitoring.
- *Locust* for load generation and benchmarking.
- Conduct test on an amd64 architecture environment.
- Master node: 2 vCPU, 4 GB RAM.
- Worker node: 4 vCPU, 8 GB RAM.
- Resource limits for microservice pods:
 - Minimum 1200m CPU and 2 GB RAM per microservice pod.

- Resource limits for Apache server pod:
 - Minimum 400m CPU and 1 GB RAM per Apache server pod.

6. DESIGN

6.1. Information Structure

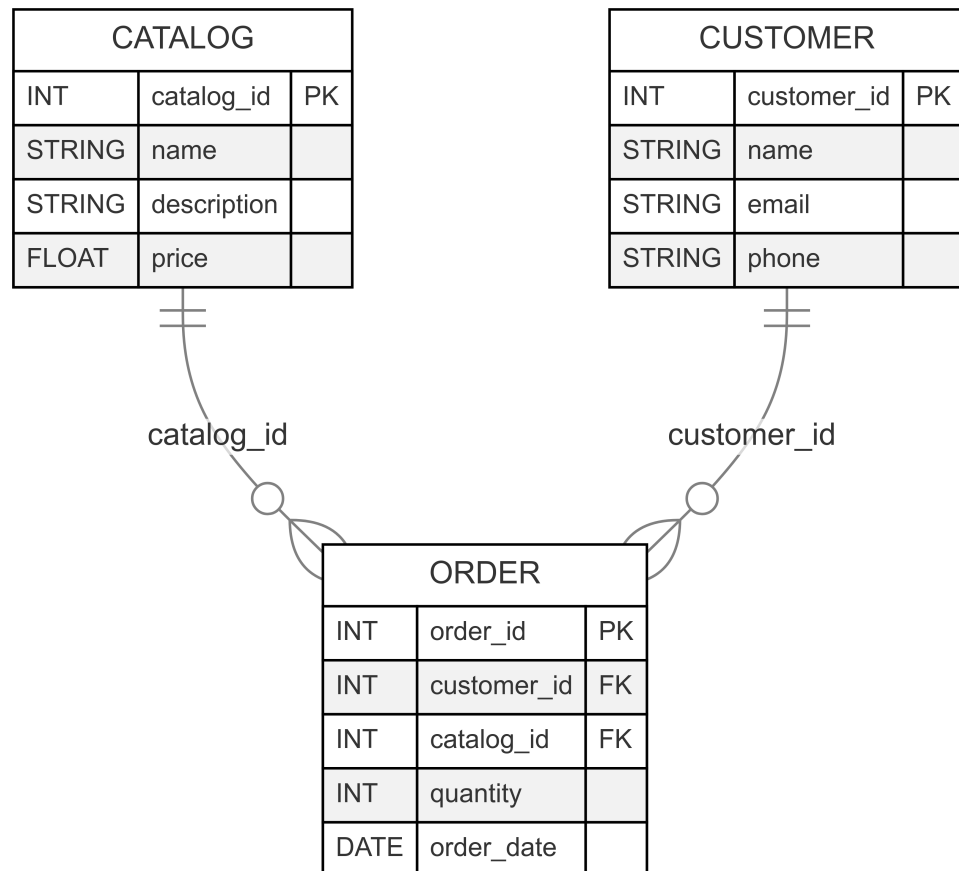


Figure 6.1. Entity-Relationship Diagram

6.2. Information Flow

6.2.1. Activity Diagram

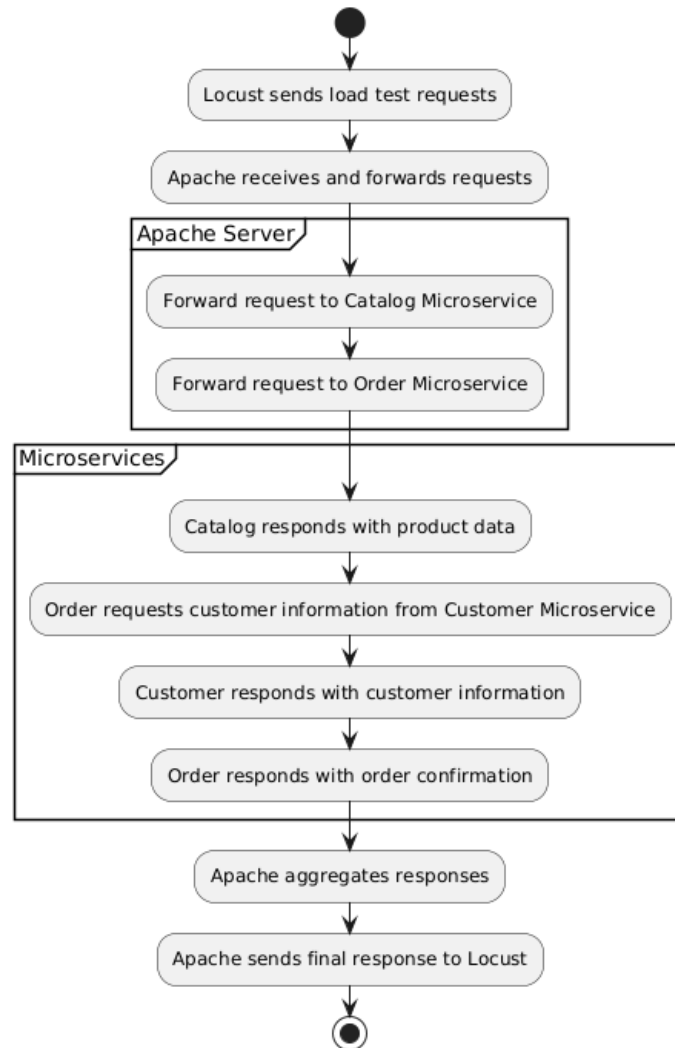


Figure 6.2. Activity Diagram

6.2.2. Sequence Diagram

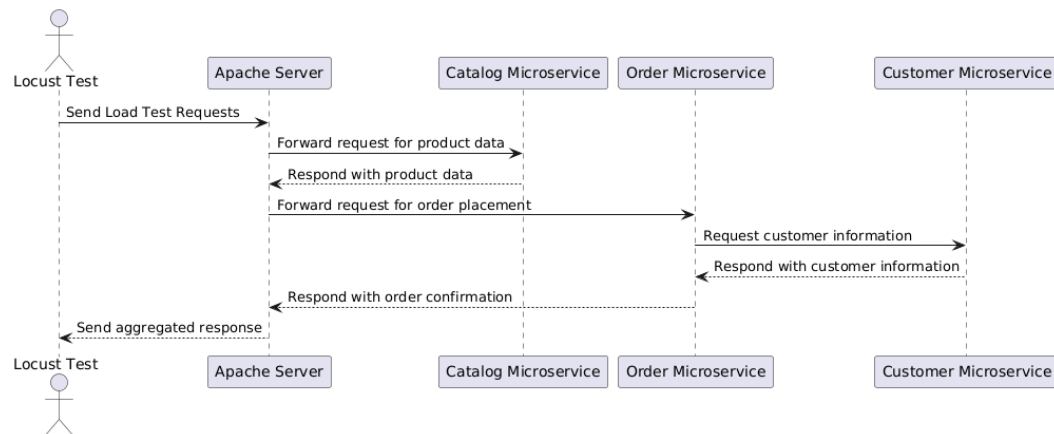


Figure 6.3. Sequence Diagram

6.3. System Design

6.3.1. Container Runtime Architectures

The following diagrams show the architecture of the three container runtimes used in the project: **runc**, **runsc** (gVisor), and **Kata Containers**.

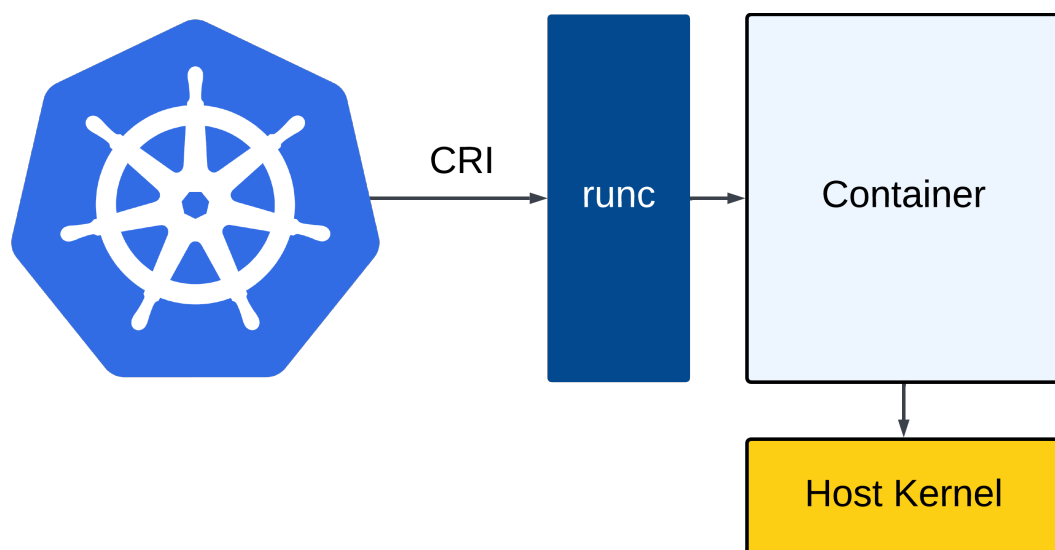


Figure 6.4. runc Architecture

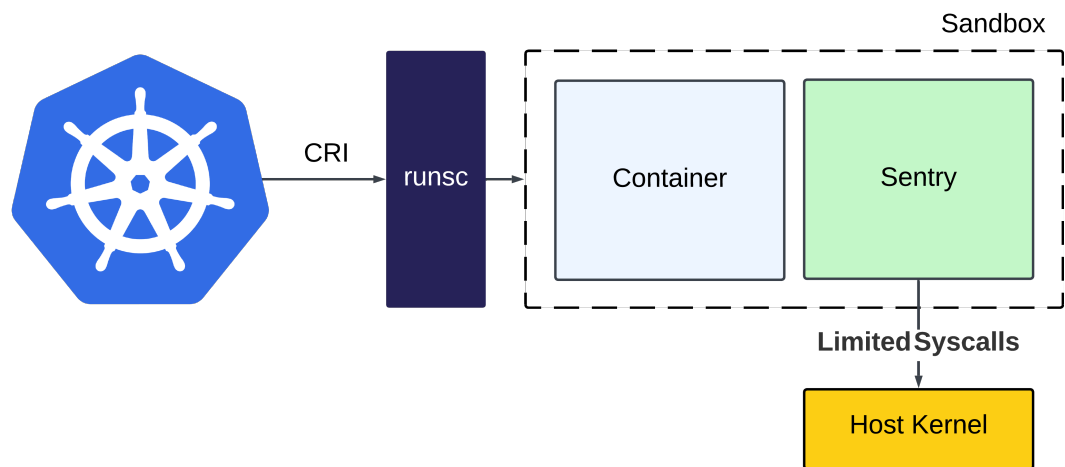


Figure 6.5. gVisor Architecture

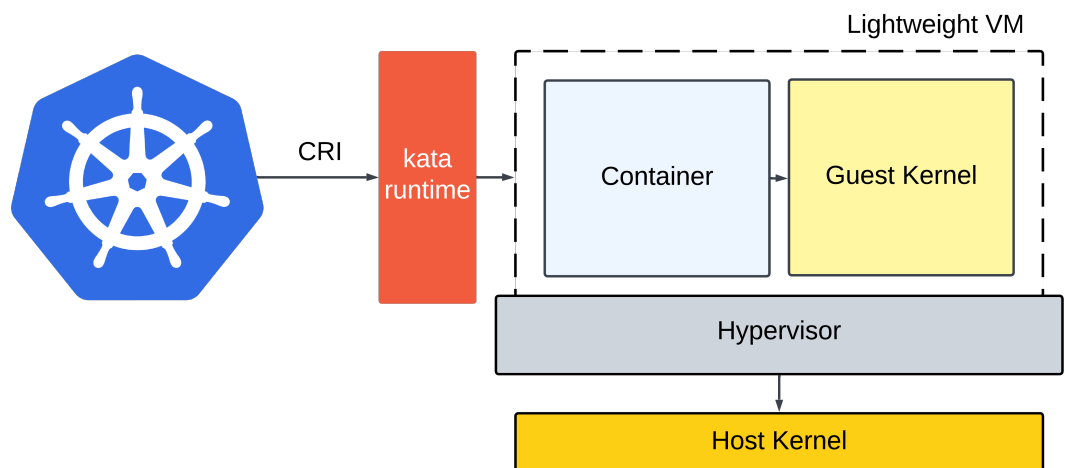


Figure 6.6. Kata Containers Architecture

6.3.2. Overall System Design

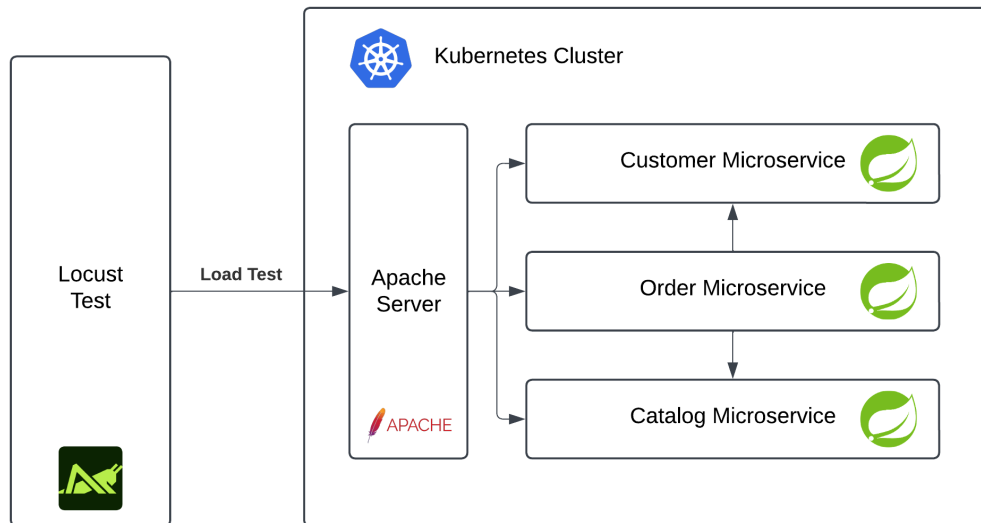


Figure 6.7. Overall System Design

7. IMPLEMENTATION AND TESTING

7.1. Implementation

This project evaluates and compares the performance and security characteristics of three container runtimes—`runc`, `runsc` (gVisor), and `Kata Containers`—in a Kubernetes environment. The objective is to assess the performance trade-offs between security and efficiency under real-world microservice workloads. The primary performance metrics analyzed include throughput, latency, resource consumption, and startup time.

7.1.1. Microservice Architecture and Stack

We have used a microservice demo application [10] from ewolff on GitHub to set up a microservice environment on Kubernetes. The system consists of an Apache server as an entry point for our application, a frontend service, and three microservices: `Order`, `Catalog`, and `Customer`. We have provided a JSON collection file for Postman integration to visualize the requests.

The microservice stack includes:

- **Frontend Service:** This apache service provides the user interface and interacts with the backend services through RESTful API calls.
- **Catalog Service:** This service exposes product catalog data to the frontend service and processes the product information.
- **Customer Service:** This service manages customer-related data, including user authentication and profile management.
- **Order Service:** This service processes customer orders and communicates with both the `Catalog` and `Customer` services for handling order details.

7.1.2. Container Runtime Setup

Each container runtime was configured as follows:

- **runc:** As the default runtime for Kubernetes, **runc** was used to run containers with minimal overhead to maximize performance.
- **runsc (gVisor):** **runsc** was configured to use a user-space kernel that intercepts system calls, providing enhanced isolation at the cost of some performance overhead.
- **Kata Containers:** **Kata Containers** was configured to run containers inside lightweight virtual machines (microVMs), providing the strongest isolation but requiring more resources and exhibiting slower startup times compared to the other runtimes.

Each runtime was tested under the same conditions to ensure a fair comparison of their performance and security characteristics.

7.2. Testing

The performance evaluation was conducted using **Locust**, a load testing tool capable of simulating user traffic for the microservices. Custom Locust scripts were written to generate HTTP requests, simulating user interactions with the system. The testing scenarios were designed to cover both low and high-load conditions, mimicking typical usage patterns as well as peak traffic loads. For more details on the test scripts, please refer to Appendix A.

7.2.1. Test Scenarios

The following test scenarios were executed:

- **Low Load Scenario:** This scenario simulates typical user traffic, with a steady

flow of requests distributed evenly across the microservices.

- **High Load Scenario:** This scenario tests the system’s performance under peak load conditions, where a large number of simultaneous users generate a burst of traffic to the services.

Pod Startup Time Measurement: To measure the startup times, we modified the Kubernetes deployment YAML files to create multiple replicas of the pods. Using a custom script, we calculated the average startup time by dividing the total time to start all replicas by the number of replicas. The results were then analyzed for each runtime.

Prometheus for Resource Monitoring: A Prometheus server was set up using Kubernetes deployment YAMLs to monitor and collect metrics like CPU usage and memory consumption during the tests. Although Kubernetes Metrics Server was initially used, it did not provide accurate results for the `gVisor` runtime due to resource consumption exceeding the node’s available resources. As a result, Prometheus was used to gather accurate CPU usage data.

For the full scripts and further details, please refer to Appendix A and the project’s GitHub repository.

7.2.2. Test Execution

Once the testing environment was set up, the Locust test scripts were executed across all three container runtimes. The tests were run for 5 minutes to gather enough data for a reliable comparison. During the testing phase, monitoring tools like Prometheus were used to gather real-time data on resource usage, and Kubernetes Metric Server was employed to track overall pod health and performance metrics.

7.3. Deployment

The deployment of the microservices was managed using Kubernetes, ensuring that the container runtimes could be switched dynamically to evaluate each runtime's impact on performance and security.

7.3.1. Kubernetes Configuration

For the testing environment, we used DigitalOcean as our Cloud Service Provider (CSP) to deploy and manage our Kubernetes cluster. The Kubernetes cluster was configured to support multiple container runtimes, namely **gVisor** (**runsc**) and **Kata Containers**. The appropriate runtime configurations were applied to the nodes within the cluster to ensure each runtime could be tested under the same set of conditions.

7.3.1.1. Container Runtime Configuration. To enable the use of **gVisor** and **Kata Containers** as alternative runtimes within the Kubernetes environment, we performed the following steps:

- **gVisor (runsc):** We configured **gVisor** as a runtime by installing and setting up the **runsc** binary on the Kubernetes nodes. A custom runtime class was created in Kubernetes for **gVisor** to allow for runtime-specific settings, such as user-space kernel isolation.
- **Kata Containers:** Similarly, **Kata Containers** was configured on the Kubernetes nodes to run containers within lightweight virtual machines (microVMs). A separate runtime class was created for **Kata Containers**, ensuring that each container deployed using this runtime would run inside a microVM for enhanced security and isolation.

Both runtime classes were defined in the `config.toml` file, which was published in our repository for transparency and reproducibility. This configuration file outlines the runtime-specific settings, such as memory limits, CPU resources, and networking

configurations.

7.3.1.2. Microservice Deployment. To test the performance and resource utilization of both container runtimes, we deployed our microservice architecture on the Kubernetes cluster. The microservice stack was composed of four independent components, each running as a separate pod. These components were distributed across the three different runtimes (`runc`, `gVisor`, and `Kata Containers`) to allow for a comprehensive comparison.

Each of the four microservice components (`frontend service`, `catalog service`, `customer service`, and `order service`) was deployed using its own Kubernetes Deployment YAML file. By creating multiple replicas of each service, we ensured that a sufficient number of pods were available for load testing. This resulted in a total of 12 pods (4 components, each with 3 runtime-specific deployments).

For communication between the microservices and for easy access to the services from external clients, we configured each service as a `NodePort`. This configuration allowed us to expose the services on specific ports, making it easier to interact with them during the testing phase, regardless of the container runtime being used.

8. RESULTS

This chapter presents the results of the performance evaluation for three container runtimes: **runc**, **runsc** (gVisor), and **Kata Containers**. The evaluation covers various performance metrics, including CPU consumption, memory usage, response time, container startup time, and throughput under low and high load conditions.

8.1. CPU Consumption

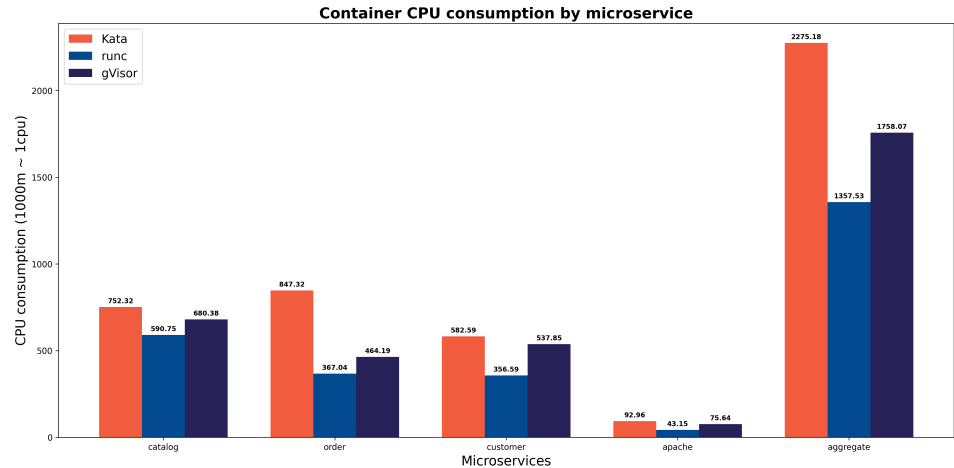


Figure 8.1. Average CPU consumption across container runtimes during load tests.

Figure 8.1 shows the average CPU usage by the microservices across different runtime environments. **runc** had the lowest CPU overhead, while **Kata Containers** consumed the most CPU due to its use of lightweight virtual machines.

8.2. Memory Usage

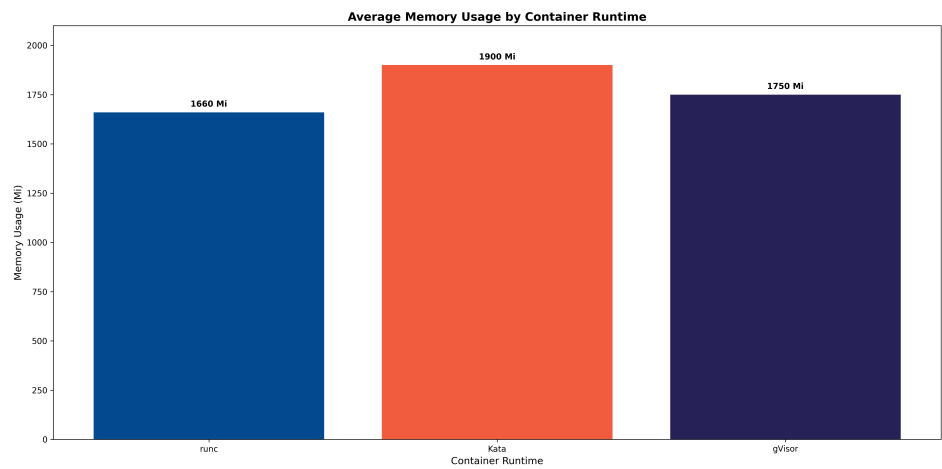


Figure 8.2. Average memory usage across container runtimes during load tests.

The memory usage comparison in Figure 8.2 highlights that **runc** had the smallest memory footprint, while **Kata Containers** required more memory due to virtualization overhead.

8.3. Response Time Comparison

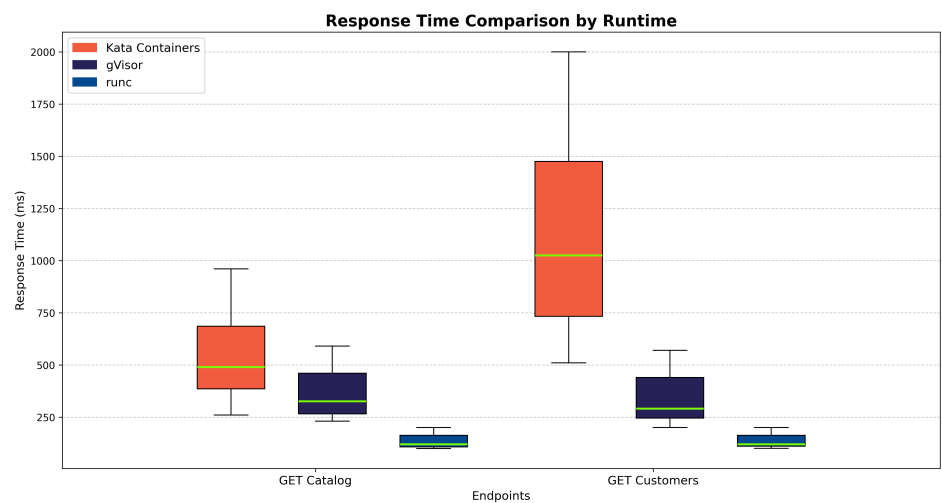


Figure 8.3. Response time comparison for Catalog and Customer endpoints.

Figure 8.3 illustrates the response time for key microservice endpoints. **runc** provided the fastest response time, followed by **runsc**, while **Kata Containers** exhibited

the highest latency.

8.4. Container Startup Time

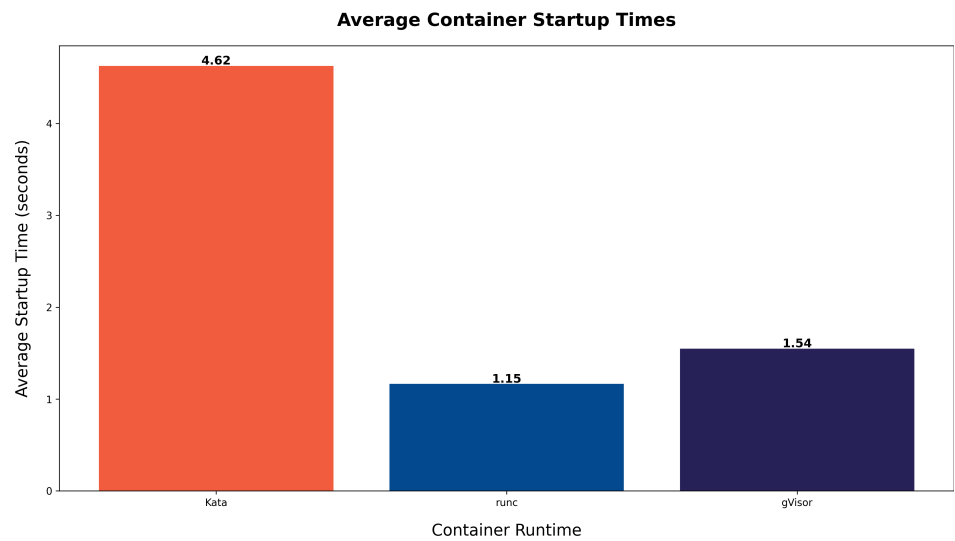


Figure 8.4. Average startup time for containers across runtimes.

Figure 8.4 compares the container startup time. **Kata Containers** showed significantly longer startup times compared to **runc** and **runsc**.

8.5. Throughput at Low Load

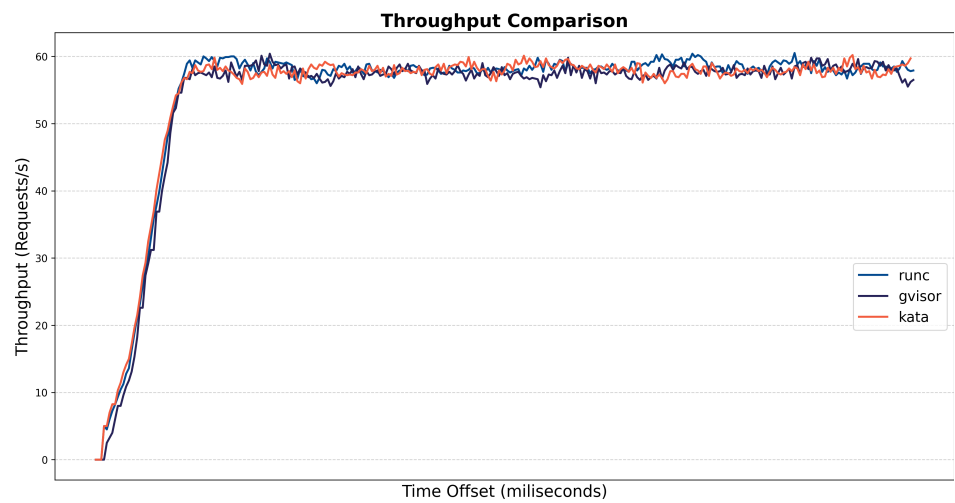


Figure 8.5. Throughput comparison under low load conditions.

Under low load conditions (Figure 8.5), all three runtimes exhibited similar throughput, indicating minimal performance overhead at light workloads.

8.6. Throughput at High Load

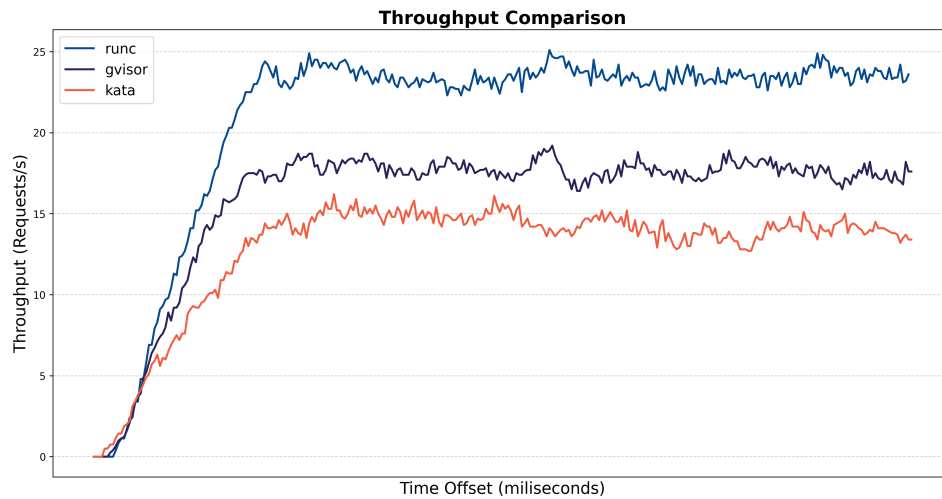


Figure 8.6. Throughput comparison under high load conditions.

At high load (Figure 8.6), **runc** maintained the highest throughput, while **runsc** and **Kata Containers** experienced performance degradation due to increased resource consumption.

8.7. Summary of Results

The results indicate that:

- **runc** provided the best overall performance with minimal resource consumption and fastest response times.
- **runsc** offered moderate performance with increased overhead due to syscall emulation, but it balanced security and performance better than **runc**.
- **Kata Containers** provided the highest level of isolation at the cost of significant performance degradation in terms of CPU, memory usage, and response time.

These results highlight the trade-offs between performance and security across different container runtimes, emphasizing the importance of selecting a runtime based on workload-specific requirements.

9. CONCLUSION

This project evaluated and benchmarked the performance of three container runtimes—**runc**, **runsc** (gVisor), and **Kata Containers**—in a Kubernetes environment, under realistic microservice workloads. The evaluation aimed to provide insights into the trade-offs between security features and performance overheads of these runtimes, particularly focusing on throughput, latency, resource usage, and startup times.

The findings from this analysis reveal that each runtime offers distinct advantages and trade-offs:

- **runc** performed the best in terms of overall throughput and latency, exhibiting minimal overhead compared to **runsc** and **Kata Containers**. As a lightweight runtime adhering to the Open Container Initiative (OCI) standards, **runc** was optimized for high-performance workloads, making it a suitable choice for environments where performance is paramount, and security concerns can be managed through other means.
- **runsc** (gVisor), provided moderate security by intercepting and emulating syscalls in user space, which added some performance overhead. While it showed slightly reduced performance compared to **runc**, it is well-suited for use cases where security is a higher priority. The impact on performance, especially in high-load scenarios, was observable but not prohibitive, making **runsc** an appealing choice for applications requiring enhanced security without the full performance hit of VM-based isolation.
- **Kata Containers** exhibited the highest level of security by using microVMs to isolate containers. While this increased the security boundaries significantly, it came at the cost of resource consumption and slower startup times, particularly under high-load conditions. The performance degradation was more pronounced compared to **runc** and **runsc**, which makes **Kata Containers** most suitable for scenarios where security is a critical concern and the workload can tolerate the

associated overhead in resource consumption and startup time.

In terms of resource usage, **runc** was the most efficient, consuming the least CPU during high-load scenarios. **runsc** required slightly more resources due to its syscall interception mechanism, while **Kata Containers** consumed the most resources, owing to its VM-based isolation approach.

The key takeaway from this analysis is the balance between performance and security. While **runc** is optimal for performance-sensitive workloads, **runsc** and **Kata Containers** provide valuable security benefits, with the former offering a moderate performance hit and the latter offering stronger isolation at the cost of higher resource usage and slower startup times. The choice of runtime should thus depend on the specific workload requirements, where performance needs to be balanced with the required level of security.

Overall, this project contributes to the growing body of knowledge regarding container runtimes and provides a reusable testing suite for replicating and extending these performance evaluations. The results can help organizations make informed decisions about selecting the right runtime based on their specific needs, whether for performance-critical applications or for those requiring enhanced isolation and security.

REFERENCES

1. Wang, X., J. Du and H. Liu, “Performance and Isolation Analysis of runC, gVisor, and Kata Containers”, *Cluster Computing*, Vol. 25, pp. 1497–1513, 2022, <https://doi.org/10.1007/s10586-021-03517-8>.
2. Anger, J., “Comparison of Container Runtimes in Kubernetes”, *Georg-August-Universität Göttingen, Institute of Computer Science*, 2024, https://hps.vi4io.org/_media/teaching/autumn_term_2023/stud/scap_jule_anger.pdf.
3. van Rijn, V. and J. S. Rellermeyer, “A Fresh Look at the Architecture and Performance of Contemporary Isolation Platforms”, *Proceedings of the 22nd ACM/IFIP International Middleware Conference, 2022*, 2021, <https://doi.org/10.48550/arXiv.2110.11462>.
4. Viktorsson, W., C. Klein and J. Tordsson, “Security-Performance Trade-offs of Kubernetes Container Runtimes”, *Proceedings of the 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2020, <https://doi.org/10.1109/MASCOTS50786.2020.9285946>.
5. gVisor Team, *Performance Guidelines by gVisor*, gVisor, 2023, https://gvisor.dev/docs/architecture_guide/performance/.
6. LLC, G., “gVisor: A User-Space Kernel for Container Isolation”, , 2018, <https://gvisor.dev/>.
7. eBPF Authors, “Cilium: Securing and Observing Network Communication in Kubernetes”, *Cloud Native Computing Foundation*, 2021, <https://cilium.io/>.
8. Sysdig, “Falco: Cloud-Native Runtime Security”, , 2019, <https://falco.org/>.

9. Security, A., “Aqua Security for Kubernetes: Comprehensive Container Security”,
 , 2020, <https://www.aquasec.com/>.
10. Ewolff, “Microservice Kubernetes”, <https://github.com/ewolff/microservice-kubernetes>, 2024, accessed: 2024-01-12.

APPENDIX A: SAMPLE APPENDIX

A.0.1. *

Project Resources including metrics, source codes, and documentation slides, are available in the project's GitHub repository. This repository serves as the central location for all components necessary for understanding, running, and analyzing the project.

Repository Link: `Halil-Ibrahim-Kasapoglu/2024-cmpe492-secure-k8s`